Формальные грамматики

Лекция 3

Формальные языки и формальные грамматики

- *Теория формальных грамматик* – раздел математической лингвистики, включающий способы описания грамматик формальных языков (формальных грамматик), методы и алгоритмы анализа принадлежности цепочек языку, а также алгоритмы перевода (трансляции) алгоритмических языков на язык машины.
- Между обычными и формальными грамматиками имеется существенное различие.
- В формальных грамматиках все утверждения формулируются с помощью небольшого числа четко определенных символов и операций. Это делает формальные грамматики очень простыми с точки зрения их логического строения и облегчает изучение их свойств и свойств, определяемых с их помощью формальных языков.
- При попытке применить формальные грамматики для естественных языков возникают трудности, обусловленные громоздкостью описаний, поэтому формальные грамматики в этом случае не способствуют повышению обозримости лингвистических описаний и неудобны для непосредственного использования человеком. Например, учить иностранный язык с помощью его формальной грамматики вряд ли целесообразно.

Формальные языки

- **Формальный язык – это математическая модель реального языка**, под которым здесь понимается некий способ коммуникации субъектов друг с другом. Для общения субъекты используют конечный набор знаков (символов), которые проговариваются (выписываются) в строгом временном порядке, т.е. образуют линейные последовательности, которые обычно называют словами или предложениями.
- Чтобы изучать языки с помощью математических методов, необходимо сначала выделить из языка его свойства, которые представляются важными для изучения, а затем эти свойства строго определить.
- Полученная таким образом абстракция будет называться формальным языком – математической моделью реального языка. Содержание конкретной математической модели зависит от того, какие свойства важны для изучения, т.е. что планируется в данный момент выделить и изучать.
- В теории формальных языков представляется важным изучить в первую очередь законы расположения слов рядом друг с другом, т.е. **синтаксические свойства текстов. Поэтому формальный язык задается как множество последовательностей, составленных из элементов конечного алфавита**.

Формальные грамматики

- **Возникает вопрос: как можно задать формальный язык?** Если язык конечен, то можно просто выписать все его фразы. Но что делать, если язык бесконечен, как его задавать?
- **Назначение грамматики – задание, описание языка**. Это описание обязательно должно быть конечным, иначе человек будет не в состоянии эту грамматику понять. Но каким образом конечное задание грамматики описывает бесконечные совокупности предложений языка? Это возможно только в том случае, если строение всех цепочек языка основано на единых принципах.
- Итак, **грамматика языка описывает законы внутреннего строения его предложений. Такие законы называют синтаксическими закономерностями.**
- Формальная грамматика – это математическая модель грамматики, описанная в рамках какой-то синтаксической теории.
- Самый **распространенный метаязык для задания грамматик – это порождающие грамматики Хомского.**

Формальные грамматики

- С алгоритмической точки зрения грамматики можно подразделить по способу задания языка.
- **распознающие** – автоматические устройства (алгоритмы), которым на вход подается цепочка языка, а на выходе автомат сообщает «Да», если цепочка принадлежит языку, и «Нет» – в противном случае; этот прием является самым распространенным в практике создания лингвистических программ;
- **порождающие** – используются для моделирования процесса порождения конкретных предложений формального языка; потенциально может быть порождено все бесконечное множество предложений языка. На практике используется для задания языков и служит основой для построения автоматических устройств.

Основные понятия формальных грамматик

- **Определение**: ***алфавит*** – это любое конечное множество символов. Рассмотрение пустых и бесконечных алфавитов лишено практического смысла.
- **Определение**: ***словом, предложением*** в некотором алфавите *A* называется произвольная цепочка конечной длины, состоящая из символов *A*.
- Например, в алфавите {0, 1} можно построить следующие предложения {0, 1, 00, 01, 10, …}.
- Цепочки составляются из символов согласно правилам синтаксиса.
- Существует цепочка, которая не содержит ни одного символа, она называется *пустой цепочкой*. Для нее вводится специальное обозначение, чаще всего используют значки λ или ε. Предполагается, что сама буква ε(λ) в алфавит не входит, она лишь помогает обозначить пустую последовательность символов в рассуждениях о языках.

Операции над цепочками

- ***Длина цепочки*** равна количеству символов в ней. Будем обозначать длину цепочки x как $|x|$, например, $|abc|= 3$, $|\varepsilon|= 0$.
- Если x и y – цепочки, то $x \circ y$ – их ***конкатенация*** (*слияние*). Для упрощения конкатенацию цепочек записывается: xy.
  Имеет место $\varepsilon x = x\varepsilon = x$.
- Для некоторой цепочки x обозначим через x^k конкатенацию k цепочек x и будем ее называть ***степенью цепочки***, $x^k = xxxxxxxx \ldots x$ (x входит k раз).
  Свойства степени цепочки: $x^0 = \varepsilon$, $x^1 = x$; $x^n = xx^{n-1} = x^{n-1}x$.
- Если x и y – цепочки и имеет место xy, то x – ***префикс цепочки*** xy;
  y – *суффикс цепочки* xy; x и y – ***подцепочки*** *цепочки* xy.
- Если A и B – множества цепочек, то $A \times B = \{ab \mid a \in A, b \in B\}$ **- *декартово произведение***.
  Каждый элемент декартова произведения множеств цепочек представляет собой цепочку, которая содержит в качестве префикса элемент множества A, а в качестве суффикса – элемент множества B.
- ***Степенью*** A^k алфавита A называется декартово произведение $A \times \ldots \times A$ (k экземпляров множества A). Например, для алфавита $A = \{0, 1\}$ $A^0 = \{\varepsilon\} = \emptyset$,
  $A^1 = \{0, 1\}$ – это сам алфавит A,
  $A^2 = \{00, 01, 10, 11\}$ – это все множество цепочек длины 2, составленное из символов алфавита A и т.д.

Формальный язык

- Пусть Σ — алфавит.
- Множество всех цепочек в алфавите Σ обозначается через Σ^*.
- Множество всех непустых цепочек в алфавите Σ обозначается через Σ^+.
- **Языком (формальным языком) в алфавите (над алфавитом) Σ называется любое подмножество множества Σ^*.**

Пример языка: скобочный язык LB

- Алфавит языка LB: $\{[,\ ]\}$.
- Индуктивное определение языка LB:
  - $\varepsilon \in LB$;
  - если $u \in LB$ и $v \in LB$, то $[u] \in LB$ и $uv \in LB$.
- Примеры:
  $[]$, $[][]$, $[[]][] \in LB$,
  $[,\ ][,\ []] \notin LB$.

Пример языка: язык списков LL

- Алфавит языка LL: $\{\mathrm{a},\ ;,\ [,\ ]\}$.
- Индуктивное определение языка LL:
  - $\mathrm{a} \in LL$;
  - если $u \in LL$ и $v \in LL$, то $[u] \in LL$ и $u; v \in LL$.
- Примеры:
  $[\mathrm{a}; \mathrm{a}],\ \mathrm{a}; [\mathrm{a}],\ [[\mathrm{a}]]; [\mathrm{a}] \in LL$,
  $[],\ [\mathrm{a}]] \notin LL$.

Пример языка: язык описаний типов LD

- Алфавит языка LD: $\{\mathtt{i},\ :,\ ;,\ \mathtt{int},\ \mathtt{real}\}$.
- Каждая цепочка языка LD имеет вид:
  - непустой список букв $\mathtt{i}$, разделённых точкой с запятой,
  - затем двоеточие,
  - наконец, один из символов $\mathtt{int}$ или $\mathtt{real}$.
- Примеры:
  $\mathtt{i} : \mathtt{real} \in LD$,
  $\mathtt{i}; \mathtt{i}; \mathtt{i} : \mathtt{int} \in LD$.

Пример языка: язык двоичных чисел LN

- Алфавит языка LN: $\{0,\ 1,\ .\}$.
- Каждая цепочка языка LN имеет вид u, $.v$ или $u.v$, где u и v — произвольные непустые цепочки в алфавите $\{0,\ 1\}$ (т. е. $u, v \in \{0,\ 1\}^+$).
- Примеры: $1,\ .0100,\ .01,\ 01.1,\ 1.1 \in LN$.
- Язык LN' получаем, если дополнительно потребуем, чтобы целая часть (u) числа начиналась с 1, дробная часть (v) заканчивалась на 1.
- Примеры:
  $1,\ .01,\ 1.1 \in LN'$,
  $.0100,\ 01.1 \notin LN'$.

Пример языка: язык арифметических выражений LA

- Алфавит языка LA: $\{\mathtt{x},\ +,\ *,\ (,\ )\}$.
- Индуктивное определение языка LA:
  - $\mathtt{x} \in LA$;
  - если $u \in LA$ и $v \in LA$, то $(u) \in LA$, $u + v \in LA$ и $u * v \in LA$.
- Примеры: $(\mathtt{x}),\ \mathtt{x} + \mathtt{x} * \mathtt{x},\ (\mathtt{x} + \mathtt{x}) * \mathtt{x} \in LA$.

Способы задания формальных языков

- **перечисление** всех допустимых цепочек языка, например, язык $L = \{ab, c, ca\}$; способ можно использовать для определения небольших по объему языков. С увеличением мощности множества L возникают естественные трудности, которые становятся непреодолимыми при стремлении мощности языка L к бесконечности;
- **описание с помощью теоретико-множественной формулы**, например, язык $L = \{a^n b^n : n \geq 0\}$ включает множество цепочек
- $\{\varepsilon, ab, aabb, aaabbb, \ldots\}$; это бесконечное множество, т.е. рассматриваемый способ не обладает недостатком первого способа, но для реальных языков не всегда удается записать соответствующую теоретико-множественную формулу.
- для решения практических задач определения языков, в том числе языков программирования, необходим способ задания в общем случае бесконечных языков с помощью записи конечного числа определений. **Формальные грамматики** обладают свойствами задания бесконечных языков с помощью конечного числа изобразительных средств, поэтому их изучение и практическое использование представляет наибольший интерес.

Представление языка с помощью формальной грамматики

Определение: *Формальной грамматикой* G называется четверка вида:

$$G = (N, T, P, S),$$

где N – конечное множество нетерминальных символов грамматики, нетерминальный алфавит; будем также использовать понятие «вспомогательный алфавит»; T – конечное множество терминальных символов грамматики, основной алфавит, алфавит определяемого языка, $N \cap T = \varnothing$; P – множество правил вывода грамматики, являющееся конечным подмножеством множества $(N \cup T)^{+} \times (N \cup T)^{*}$; элемент (α, β) множества P называется *правилом вывода* или *правилом подстановки* и записывается в виде $\alpha \rightarrow \beta$ (читается: «из цепочки α выводится цепочка β»); S – начальный символ грамматики, $S \in N$.

Представление языка с помощью формальной грамматики

- Базовые элементы, из которых конструируются строки и предложения определяемого формальной грамматикой языка, называются **терминальными символами** языка.
- На завершающем шаге вывода следует получить строку, которая состоит из терминального алфавита, т.е. алфавита, определяемого грамматикой языка. Такие строки получили в формальных грамматиках название ***предложение***.
- **Нетерминальные символы** грамматики есть синтаксические переменные – символы, не присутствующие в построенном с помощью вывода предложении, но принимающие участие в определении грамматики и построении промежуточных в выводе строк.

Пример: условный оператор

- При объяснении **синтаксиса условного оператора** языка программирования можно встретить строку: **if (*Y*) *O*.**
- Далее идут пояснения о том, что ключевое слово **"if" и скобки "(" и ")"** являются обязательными в структуре оператора словами. В нашем случае можно сказать, что эти слова являются **терминальными цепочками**, т.е. словами определяемого языка.
- В дальнейших пояснениях к условному оператору увидим, что символами "*Y*" и "*O*" обозначены, соответственно, условное выражение и оператор, которые должны появиться в условном операторе на месте символов **"*Y*" и "*O*".**
  В нашем случае можно сказать, что "*Y*" и "*O*" – это синтаксические переменные или **нетерминальные символы**, которые могут участвовать в подстановке на их место конкретных конструкций.
- Естественно, что в грамматике должны быть определены **правила подстановки**, применяемые в процессе вывода и приводящие, в конечном итоге, к **терминальной цепочке**, например, **if (*a* > *b*) *b* = *a*.**

Операция подстановки

- Основная цель всех грамматик заключается в описании того, как **конструируются строки терминальных символов**.
- **Основной операц**ией при таком построении является ***подстановка***.
- Предположим, что имеется строка, содержащая синтаксическую переменную.
  Операция подстановки произведет замену этой переменной в строке на подстроку, соответствующую переменной.
- Для записи операции подстановки используются **правила вывода** – элементы множества *P*.

Представление языка с помощью формальной грамматики

Определение: *Формальной грамматикой* G называется четверка вида:

$$G = (N, T, P, S),$$

где N – конечное множество нетерминальных символов грамматики, нетерминальный алфавит; будем также использовать понятие «вспомогательный алфавит»; T – конечное множество терминальных символов грамматики, основной алфавит, алфавит определяемого языка, $N \cap T = \varnothing$; P – множество правил вывода грамматики, являющееся конечным подмножеством множества $(N \cup T)^{+} \times (N \cup T)^{*}$; элемент (α, β) множества P называется *правилом вывода* или *правилом подстановки* и записывается в виде $\alpha \rightarrow \beta$ (читается: «из цепочки α выводится цепочка β»); S – начальный символ грамматики, $S \in N$.

S – начальный символ формальной грамматики

- $G = (N, T, P, S)$.
- Вывод предложений формального языка всегда начинается с начального символа грамматики.
- Это означает, что множество правил P грамматики обязательно должно включать правило вида $S \rightarrow \alpha$, где S – начальный символ грамматики.

Пример грамматики

Рассмотрим пример грамматики $G_{2.1}$:

$$G = (\{S, A\}, \{0, 1\}, \{S \to 0A1, 0A \to 00A1, A \to \varepsilon\}, S),$$

где $\{S, A\}$ – нетерминальный, вспомогательный алфавит; $\{0, 1\}$ – терминальный алфавит, алфавит определяемого языка; $\{S \to 0A1, 0A \to 00A1, A \to \varepsilon\}$ – множество правил вывода; S – начальный символ грамматики.

С помощью грамматики $G_{2.1}$ можно построить вывод предложения:

$$S \Rightarrow 0A1 \Rightarrow 00A11 \Rightarrow 000A111 \Rightarrow 000111,$$

где двойной стрелкой $\Rightarrow$ обозначена операция вывода с помощью правила грамматики.

Данная грамматика определяет язык $L = \{0^n1^n / n > 0\}$.

Правила записи

Пусть $G = (N, T, P, S)$ – грамматика. Для сокращения записи ее и введем ряд договоренностей:

1) для записи символов нетерминального алфавита N будем использовать заглавные буквы;

2) для записи символов терминального алфавита T будем использовать строчные буквы, цифры, специальные знаки +, *, (, ? и т.д.;

3) правило с начальным символом грамматики S будем записывать всегда первым в последовательности правил вывода;

4) если среди правил вывода имеется несколько правил с одинаковой левой частью, например, $U \to 0W$, $U \to 1V$, то группу таких правил будем записывать сокращенно, однократно выписывая строку из левой части правила, а различные правые части будем перечислять, разделяя символом | (или). Для приведенного примера правила запишутся так: $U \to 0W|0V$. Заметим, что здесь записаны два правила.

Примеры грамматик:

а) $S \to \varepsilon \mid 0S1$;

б) $S \to \varepsilon \mid (S) \mid SS$.

Выводимая цепочка

Определим понятие *выводимой цепочки*:

• S (начальный символ грамматики) – выводимая цепочка;

• если $\alpha\beta\gamma$ – выводимая цепочка и в грамматике имеется правило:

$$\beta \to \delta,$$

то $\alpha\delta\gamma$ – выводимая цепочка.

Обозначим операцию выводимости знаком $\Rightarrow$. Тогда понятие выводимой цепочки можно записать следующим образом: если $\alpha\beta\gamma$ – выводимая цепочка, в грамматике имеется правило $\beta \to \delta$ и имеет место $\alpha\beta\gamma \Rightarrow \alpha\delta\gamma$, то $\alpha\delta\gamma$ – выводимая цепочка.

Сентенциальная форма

Определение. Сентенциальной формой будем называть начальный символ грамматики и любую выводимую из начального символа цепочку. Для обозначения сентенциальной формы будем использовать операцию выводимости, дополненную символами * или +, т.е. $\Rightarrow^*$ или $\Rightarrow +$. Операция $\Rightarrow^*$ кратко обозначает вывод, полученный за ноль или более шагов вывода. Соответственно операция $\Rightarrow +$ обозначает вывод за один и более шагов. Рассмотрим пример. Пусть дана грамматика $S \rightarrow \varepsilon|(S)|S$ Тогда формула $S \Rightarrow^* \alpha$ обозначает множество выводимых из начального символа за ноль или более шагов грамматики цепочек

α = {S, ε, (S), (), SS, (S)S, ()(), …}

Применение правил ФГ

Подчеркнем, что формальная грамматика не является алгоритмом: правила подстановки – это не последовательность предписаний, а совокупность разрешений. Это означает: во-первых, правило вида $A \to \alpha$ понимается в грамматике как «A можно заменить на α» (а можно и не заменять), тогда как в алгоритме $A \to \alpha$ означало бы «A следует заменить на α» (нельзя не заменить); во-вторых, порядок применения правил в грамматике произволен, тогда как в алгоритме жестко задается порядок применения отдельных инструкций. Однако необходимо различать «разрешение применить правило» и «возможность применить правило», так как правило применяется только к такой цепочке, которая содержит вхождение левой части правила. Так, правило $AB \to CD$ можно применить к цепочке $AAEABC$, но нельзя применить к цепочке $AAEEBC$. Поэтому фактически порядок применения правил может диктоваться ими самими. Никаких внешних ограничений на порядок применения правил не существует.

Синтез текста с помощью формальных грамматик

Задачу *синтеза текста* можно определить как вывод из начального символа грамматики конкретного предложения определяемого языка.

Механизм распознавания лежит в основе задачи анализа текста. Распознаватель, по сути, является процедурой специального вида, которая по заданной цепочке определяет, принадлежит ли она языку. Если принадлежит, то процедура останавливается с ответом «да», т.е. *допускает* цепочку; иначе – останавливается с ответом «нет» или зацикливается. Язык, определяемый распознавателем, – это множество всех цепочек, которые он допускает. Грамматики тесно связаны с распознающими автоматическими устройствами (автоматами) и служат основой построения компонентов трансляторов.

Анализ текста с помощью формальных грамматик

Задача анализа текста представляет собой процесс, обратный задаче синтеза: для предъявленного предложения необходимо восстановить процесс вывода этого предложения с помощью грамматики. Анализ предложения можно определить как *редукцию предложения* языка к начальному символу грамматики. Задача редукции предложения может быть решена с помощью автоматического устройства (процедуры). Для краткости автоматическое устройство часто называют автоматом.

Стратегии вывода

Стратегии вывода и редукции предложений языка. Существующие алгоритмы распознавания требуют определенной дисциплины вывода цепочек: вывод бывает лево- или правосторонний.

Правосторонний вывод получается, когда подстановка осуществляется всегда в самый правый нетерминальный символ.

Правосторонний вывод

<u>Пример.</u> Дана грамматика $G_{2.2}$:

$S \to AB$

$A \to CaC$

$B \to CbC$

$C \to c.$

Тогда для нее можно построить такой правосторонний вывод:

$$\underline{S} \Rightarrow A\underline{B} \Rightarrow ACb\underline{C} \Rightarrow A\underline{C}bc \Rightarrow \underline{A}cbc \Rightarrow Ca\underline{C}cbc \Rightarrow$$
$$\Rightarrow \underline{C}accbc \Rightarrow caccbc.$$

Левосторонний вывод: подстановка осуществляется в самый левый нетерминальный символ

<u>Пример.</u> Дана грамматика $G_{2.2}$:

$S \to AB$

$A \to CaC$

$B \to CbC$

$C \to c.$

$$\underline{S} \Rightarrow \underline{A}B \Rightarrow \underline{C}acB \Rightarrow cac\underline{B} \Rightarrow cac\underline{C}bC \Rightarrow caccb\underline{C} \Rightarrow caccbc.$$